

ГУАП

КАФЕДРА № 44

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель,
должность, уч. степень, звание

подпись, дата

Н.А. Соловьева
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №5

Алгоритмы сортировки

по курсу: Алгоритмы и структуры данных

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4341В

подпись, дата

Р.С. Кулаков
инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Целью работы является изучение алгоритмов внутренней сортировки и получение практических навыков их использования и анализа их сложности.

2 Вариант

2.1 Вариант № 11

2.2 Задание

Найти количество повторяющихся чисел среди элементов массива

2.3 Алгоритм сортировки

Подсчётом

3 Описание алгоритма

Для нахождения количества повторяющихся чисел среди элементов массива A необходимо упорядочить (отсортировать) массив, а затем, проходя по отсортированному массиву, подсчитать, сколько различных чисел встречается более одного раза. Алгоритм сортировки выбирается в соответствии с вариантом задания — сортировка подсчётом

Алгоритм сортировки подсчётом выполняется в три этапа:

1. Нахождение минимального и максимального значений массива для определения диапазона значений $k = \max - \min + 1$.
2. Подсчёт количества вхождений каждого значения во вспомогательный массив счётчиков `count` размером k .
3. Восстановление отсортированного массива путём последовательной записи каждого значения столько раз, сколько зафиксировано в `count`.

Программу, реализующую алгоритм сортировки, необходимо модифицировать для подсчёта выполняемых операций сравнения и перестановок. Модификация осуществляется путём введения в текст программы специальных переменных-счётчиков `numOfCompare` и `numOfReshuffle`, которые увеличиваются при выполнении соответствующих операций.

4 Листинг программы

```
#include <iostream>
#include <vector>
using namespace std;

// Глобальные счётчики операций
int numOfCompare = 0; // количество сравнений
int numOfReshuffle = 0; // количество перестановок (записей)

// Вывод массива
void printArray(const vector<int>& arr) {
    cout << "Array: [";
    for (int i = 0; i < (int)arr.size(); i++) {
        cout << arr[i];
        if (i < (int)arr.size() - 1) cout << ", ";
    }
    cout << "]" << endl;
}

// Сортировка подсчётом
void countingSort(vector<int>& arr) {
    if (arr.empty()) return;

    // Находим минимум и максимум
    int minVal = arr[0];
    int maxVal = arr[0];
    for (int i = 1; i < (int)arr.size(); i++) {
        numOfCompare++;
        if (arr[i] < minVal) minVal = arr[i];
        numOfCompare++;
        if (arr[i] > maxVal) maxVal = arr[i];
    }

    // Создаём массив счётчиков
    int range = maxVal - minVal + 1;
    vector<int> count(range, 0);

    // Считаем вхождения каждого элемента
    for (int i = 0; i < (int)arr.size(); i++) {
        count[arr[i] - minVal]++;
    }
}
```

```

// Восстанавливаем отсортированный массив
int idx = 0;
for (int i = 0; i < range; i++) {
    while (count[i] > 0) {
        arr[idx] = minVal + i;
        numOfReshuffle++;
        idx++;
        count[i]--;
    }
}

// Поиск количества повторяющихся чисел
// (тех, что встречаются более одного раза)
int findRepeatingCount(const vector<int>& arr) {
    int repeatingCount = 0;
    int i = 0;
    while (i < (int)arr.size()) {
        int j = i + 1;
        while (j < (int)arr.size() && arr[j] == arr[i]) {
            j++;
        }
        if (j - i > 1) {
            repeatingCount++;
        }
        i = j;
    }
    return repeatingCount;
}

// Поиск по позиции
void searchByPosition(const vector<int>& arr, int pos) {
    if (pos < 0 || pos >= (int)arr.size()) {
        cout << "Error: position out of range!" << endl;
        return;
    }
    cout << "Element at position " << pos << " -> " << arr[pos] << endl;
}

// Поиск по значению
void searchByValue(const vector<int>& arr, int val) {

```

```

bool found = false;
for (int i = 0; i < (int)arr.size(); i++) {
    if (arr[i] == val) {
        cout << "Value " << val << " found at position " << i << endl;
        found = true;
    }
}
if (!found)
    cout << "Value " << val << " not found." << endl;
}

// Добавить элемент и пересортировать
void addElement(vector<int>& arr, int val) {
    arr.push_back(val);
    numOfCompare = 0;
    numOfReshuffle = 0;
    countingSort(arr);
    cout << "Element " << val << " added. Array re-sorted." << endl;
    cout << "Comparisons: " << numOfCompare
        << " | Placements: " << numOfReshuffle << endl;
}

// Удалить первое вхождение элемента и пересортировать
void removeElement(vector<int>& arr, int val) {
    for (auto it = arr.begin(); it != arr.end(); ++it) {
        if (*it == val) {
            arr.erase(it);
            numOfCompare = 0;
            numOfReshuffle = 0;
            countingSort(arr);
            cout << "Element " << val << " removed. Array re-sorted." << endl;
            cout << "Comparisons: " << numOfCompare
                << " | Placements: " << numOfReshuffle << endl;
            return;
        }
    }
    cout << "Element " << val << " not found." << endl;
}

int main() {
    cout << "=== Lab 5: Sorting Algorithms ===" << endl;
    cout << "Variant 11 | Algorithm: Counting Sort" << endl;
}

```

```

cout << "Task: Find the count of repeating numbers" << endl;

vector<int> arr = {5, 3, 8, 3, 1, 5, 7, 2, 8, 8, 4};

cout << endl << "Initial ";
printArray(arr);

bool isSorted = false;
int choice = -1;

while (choice != 0) {
    cout << endl;
    cout << "===== MENU =====" << endl;
    cout << "1. Show array" << endl;
    cout << "2. Sort array" << endl;
    cout << "3. Find repeating numbers" << endl;
    cout << "4. Search by position" << endl;
    cout << "5. Search by value" << endl;
    cout << "6. Add element" << endl;
    cout << "7. Remove element" << endl;
    cout << "0. Exit" << endl;
    cout << "===== " << endl;
    cout << "Your choice: ";
    cin >> choice;

    switch (choice) {
        case 1:
            printArray(arr);
            break;
        case 2:
            numOfCompare = 0;
            numOfReshuffle = 0;
            countingSort(arr);
            isSorted = true;
            cout << "Array sorted." << endl;
            printArray(arr);
            cout << "Comparisons: " << numOfCompare
                << " | Placements: " << numOfReshuffle << endl;
            break;
        case 3:
            if (!isSorted) {
                numOfCompare = 0;

```

```

        numOfReshuffle = 0;
        countingSort(arr);
        isSorted = true;
    }
    cout << "Number of repeating values: "
        << findRepeatingCount(arr) << endl;
    break;
case 4: {
    int pos;
    cout << "Enter position: ";
    cin >> pos;
    searchByPosition(arr, pos);
    break;
}
case 5: {
    int val;
    cout << "Enter value: ";
    cin >> val;
    searchByValue(arr, val);
    break;
}
case 6: {
    int val;
    cout << "Enter value to add: ";
    cin >> val;
    addElement(arr, val);
    isSorted = true;
    printArray(arr);
    break;
}
case 7: {
    int val;
    cout << "Enter value to remove: ";
    cin >> val;
    removeElement(arr, val);
    isSorted = true;
    printArray(arr);
    break;
}
case 0:
    cout << "Goodbye!" << endl;
    break;

```

```

        default:
            cout << "Unknown command." << endl;
            break;
        }
    }
    return 0;
}

```

4.1 Листинг на Kotlin

```

import java.util.Scanner

// Глобальные счётчики операций
var numOfCompare = 0    // количество сравнений
var numOfReshuffle = 0  // количество перестановок (записей)

// Вывод массива
fun printArray(arr: MutableList<Int>) {
    println("Array: [{
        arr.joinToString(", ")
    }])")
}

// Сортировка подсчётом
fun countingSort(arr: MutableList<Int>) {
    if (arr.isEmpty()) return

    // Находим минимум и максимум
    var minVal = arr[0]
    var maxVal = arr[0]

    for (i in 1 until arr.size) {
        numOfCompare++
        if (arr[i] < minVal) {
            minVal = arr[i]
        }

        numOfCompare++
        if (arr[i] > maxVal) {
            maxVal = arr[i]
        }
    }
}

```



```

// Создаём массив счётчиков
val range = maxVal - minVal + 1
val count = IntArray(range)

// Считаем количество вхождений
for (value in arr) {
    count[value - minVal]++
}

// Восстанавливаем отсортированный массив
var idx = 0

for (i in 0 until range) {
    while (count[i] > 0) {
        arr[idx] = minVal + i
        numOfReshuffle++
        idx++
        count[i]--
    }
}

// Поиск количества повторяющихся чисел
fun findRepeatingCount(arr: MutableList<Int>): Int {
    var repeatingCount = 0
    var i = 0

    while (i < arr.size) {
        var j = i + 1

        while (j < arr.size && arr[j] == arr[i]) {
            j++
        }

        if (j - i > 1) {
            repeatingCount++
        }

        i = j
    }

    return repeatingCount
}

```

```
}
```

```
// Поиск по позиции
```

```
fun searchByPosition(arr: MutableList<Int>, pos: Int) {  
    if (pos < 0 || pos >= arr.size) {  
        println("Error: position out of range!")  
        return  
    }  
  
    println("Element at position $pos -> ${arr[pos]}")  
}
```

```
// Поиск по значению
```

```
fun searchByValue(arr: MutableList<Int>, value: Int) {  
    var found = false  
  
    for (i in arr.indices) {  
        if (arr[i] == value) {  
            println("Value $value found at position $i")  
            found = true  
        }  
    }  
  
    if (!found) {  
        println("Value $value not found.")  
    }  
}
```

```
// Добавить элемент и пересортировать
```

```
fun addElement(arr: MutableList<Int>, value: Int) {  
    arr.add(value)  
  
    numOfCompare = 0  
    numOfReshuffle = 0  
  
    countingSort(arr)  
  
    println("Element $value added. Array re-sorted.")  
    println("Comparisons: $numOfCompare | Placements: $numOfReshuffle")  
}
```

```
// Удалить первое вхождение элемента и пересортировать
```

```

fun removeElement(arr: MutableList<Int>, value: Int) {
    val removed = arr.remove(value)

    if (removed) {
        numOfCompare = 0
        numOfReshuffle = 0

        countingSort(arr)

        println("Element $value removed. Array re-sorted.")
        println("Comparisons: $numOfCompare | Placements: $numOfReshuffle")
    } else {
        println("Element $value not found.")
    }
}

fun main() {
    val scanner = Scanner(System.`in`)

    println("=== Lab 5: Sorting Algorithms ===")
    println("Variant 11 | Algorithm: Counting Sort")
    println("Task: Find the count of repeating numbers")

    val arr = mutableListOf(5, 3, 8, 3, 1, 5, 7, 2, 8, 8, 4)

    println("\nInitial ")
    printArray(arr)

    var isSorted = false
    var choice: Int

    do {
        println()
        println("===== MENU =====")
        println("1. Show array")
        println("2. Sort array")
        println("3. Find repeating numbers")
        println("4. Search by position")
        println("5. Search by value")
        println("6. Add element")
        println("7. Remove element")
        println("0. Exit")
    }
}

```

```

println("=====")
print("Your choice: ")

choice = scanner.nextInt()

when (choice) {
    1 -> {
        printArray(arr)
    }

    2 -> {
        numOfCompare = 0
        numOfReshuffle = 0

        countingSort(arr)
        isSorted = true

        println("Array sorted.")
        printArray(arr)

        println(
            "Comparisons: $numOfCompare | Placements: $numOfReshuffle"
        )
    }

    3 -> {
        if (!isSorted) {
            numOfCompare = 0
            numOfReshuffle = 0

            countingSort(arr)
            isSorted = true
        }
        println(
            "Number of repeating values: ${
                findRepeatingCount(arr)
            }"
        )
    }

    4 -> {
        print("Enter position: ")
        val pos = scanner.nextInt()
    }
}

```

```

        searchByPosition(arr, pos)
    }

    5 -> {
        print("Enter value: ")
        val value = scanner.nextInt()

        searchByValue(arr, value)
    }

    6 -> {
        print("Enter value to add: ")
        val value = scanner.nextInt()

        addElement(arr, value)
        isSorted = true

        printArray(arr)
    }

    7 -> {
        print("Enter value to remove: ")
        val value = scanner.nextInt()

        removeElement(arr, value)
        isSorted = true

        printArray(arr)
    }

    0 -> {
        println("Goodbye!")
    }

    else -> {
        println("Unknown command.")
    }
}

} while (choice != 0)
}

```

5 Контрольный пример

Исходный массив A (n = 11 элементов):

$A = \{ 5, 3, 8, 3, 1, 5, 7, 2, 8, 8, 4 \}$

– Шаг 1. Нахождение минимума и максимума:

$\min = 1, \max = 8, \text{ диапазон } k = 8 - 1 + 1 = 8$

– Шаг 2. Подсчёт вхождений (индекс = значение - min):

Значение: 1 2 3 4 5 6 7 8

count[i]: 1 1 2 1 2 0 1 3

– Шаг 3. Восстановление массива:

Отсортированный A = $\{ 1, 2, 3, 3, 4, 5, 5, 7, 8, 8, 8 \}$

– Шаг 4. Подсчёт повторяющихся чисел:

- 1 встречается 1 раз → не повторяется
- 2 встречается 1 раз → не повторяется
- 3 встречается 2 раза → ПОВТОРЯЕТСЯ
- 4 встречается 1 раз → не повторяется
- 5 встречается 2 раза → ПОВТОРЯЕТСЯ
- 7 встречается 1 раз → не повторяется
- 8 встречается 3 раза → ПОВТОРЯЕТСЯ

Результат: количество повторяющихся чисел = 3 (это числа 3, 5 и 8)

```
=== Lab 5: Sorting Algorithms ===
Variant 11 | Algorithm: Counting Sort
Task: Find the count of repeating numbers

Initial Array: [5, 3, 8, 3, 1, 5, 7, 2, 8, 8, 4]

===== MENU =====
1. Show array
2. Sort array
3. Find repeating numbers
4. Search by position
5. Search by value
6. Add element
7. Remove element
0. Exit
=====
```

Рисунок 1 – Запуск программы

```
=====
Your choice: 2
Array sorted.
Array: [1, 2, 3, 3, 4, 5, 5, 7, 8, 8, 8]
Comparisons: 20 | Placements: 11
```

Рисунок 2 – Сортировка массива

```
=====
Your choice: 3
Number of repeating values: 3
```

Рисунок 3 – Подсчет повторяющихся значений

```
=====
Your choice: 4
Enter position: 5
Element at position 5 -> 5
```

Рисунок 4 – Поиск значения по индексу

```
=====
Your choice: 6
Enter value to add: 10
Element 10 added. Array re-sorted.
Comparisons: 22 | Placements: 12
Array: [1, 2, 3, 3, 4, 5, 5, 7, 8, 8, 8, 10]
```

Рисунок 5 – Добавление элемента с пересортировкой

```
=====
Your choice: 5
Enter value: 10
Value 10 found at position 11
```

Рисунок 6 – Поиск позиции элемента по значению

```
=====
Your choice: 7
Enter value to remove: 8
Element 8 removed. Array re-sorted.
Comparisons: 20 | Placements: 11
Array: [1, 2, 3, 3, 4, 5, 5, 7, 8, 8, 10]
```

Рисунок 7 – Удаление значения из массива

6 Количество операций сравнения и перестановок

При сортировке массива A из $n = 11$ элементов с диапазоном $k = 8$:

6.1 Операции сравнения (numOfCompare):

В цикле поиска min/max выполняется по 2 сравнения для каждого из $(n - 1) = 10$ элементов:

$$\text{numOfCompare} = 2 * (n - 1) = 2 * 10 = 20 \text{ сравнений}$$

6.2 Операции перестановок / записей (numOfReshuffle):

При восстановлении массива каждый из $n = 11$ элементов записывается ровно один раз:

$$\text{numOfReshuffle} = n = 11 \text{ записей}$$

6.3 Фактические значения, полученные при выполнении программы:

Comparisons: 20

Placements: 11

7 Временная и пространственная сложность алгоритма

Разработанный алгоритм использует следующие данные:

- исходный массив размером n элементов типа `int`
- вспомогательный массив счётчиков `count` размером $k = \text{max} - \text{min} + 1$
- переменные `minVal`, `maxVal`, `range`, `idx` типа `int` (константное число переменных)

7.1 Пространственная сложность:

$$v = n * C_{\text{int}} + k * C_{\text{int}} + \text{const} * C_{\text{int}}$$

- где C_{int} — константа, характеризующая объём памяти под переменную
- целого типа; k — диапазон значений массива.

$$V(n) = O(v) = O(\max(O(n * C_int), O(k * C_int), O(1))) = O(\max(O(n), O(k))) \\ = O(n + k)$$

Если диапазон k не зависит от n и считается константой, то $V(n) = O(n)$

7.2 Временная сложность:

Алгоритм состоит из четырёх последовательных этапов:

1. Поиск min и max: $T1 = O(n)$
2. Заполнение count[]: $T2 = O(n)$
3. Восстановление массива: $T3 = O(n + k)$
4. Поиск повторяющихся: $T4 = O(n)$

$$T_итого = O(\max(O(n), O(n), O(n + k), O(n))) = O(n + k)$$

Если диапазон k не зависит от n и считается константой, то $T(n) = O(n)$. Таким образом, сортировка подсчётом является алгоритмом линейной временной сложности $O(n)$, что выгодно отличает её от алгоритмов на основе сравнений (пузырёк, вставка и т.д.) со сложностью $O(n^2)$. Однако это достигается за счёт дополнительной памяти $O(k)$ на хранение массива счётчиков.

8 Выводы

В ходе выполнения лабораторной работы был реализован алгоритм сортировки подсчётом (Counting Sort) на языке C++ для решения задачи нахождения количества повторяющихся чисел среди элементов массива.

Программа обеспечивает:

- сортировку массива методом подсчёта с фиксацией числа сравнений и перестановок;
- поиск элемента по порядковой позиции и по значению;
- добавление и удаление элемента с последующей автоматической пересортировкой.

На контрольном примере из 11 элементов было выполнено 20 сравнений и 11 записей. Алгоритм обнаружил 3 повторяющихся числа: 3, 5 и 8. Теоретическая временная сложность алгоритма составляет $O(n + k)$, что при фиксированном диапазоне значений даёт линейную сложность $O(n)$. Пространственная сложность также равна $O(n + k)$. Алгоритм сортировки подсчётом эффективен при небольшом диапазоне целочисленных значений k относительно размера массива n . При большом диапазоне (например, $k \gg n$) расход памяти на массив счётчиков становится нецелесообразным, что является основным недостатком данного алгоритма.